Artificial Intelligence

Project Report

EMAAN IMRAN 6/1/26 LAB

1 CONTENTS

2 PROJECT TITLE

The Title of my Project is **"Mental Health Chatbot using Machine Learning".**

Dataset: https://www.kaggle.com/datasets/nguyenletruongthien/mental-health/data

2.1 INTRODUCTION

Mental illness is one of the most crucial issues these days. Individuals regardless of age or ethnicity are experiencing mental health issues such as anxiety, depression, stress and other emotional problems in life. However, due to the lack of immediate professional help, there is a need for artificial intelligence-based solutions that will guide these people about the mental health state. With the help of AI and ML techniques, a system can be created which can give initial suggestions and recommendations regarding mental health.

In this project, we aim at creating a Mental Health Chatbot using machine learning techniques. The chatbot will take text input from users and recognize whether it belongs to any particular type of mental health condition or emotion expressed by the user. In light of the recognition of categories, chatbots will give appropriate replies. This project aims to show how machine learning and NLP techniques are used for detecting mental health sentiments.

The project involves data preprocessing, feature extraction, model training, performance evaluation, and comparison of multiple machine learning algorithms. Various classification models such as Logistic Regression, Naive Bayes, Random Forest, and Support Vector Machine (SVM) will be trained and evaluated to determine the most effective model for mental health text classification.

3 DATASET DESCRIPTION

Mentioned Mental Health Conversational AI dataset was preprocessed prior to training the machine learning models. The data set was first checked for missing values and duplicate data. Values were cleaned by removing missing data and discarding duplicate data.

The data set consists of written dialogues, which are not directly usable for machine learning algorithms. So, TF-IDF (Term Frequency-Inverse Document Frequency) technique was used to convert the text data into numerical features. TF - IDF assigns weighted numerical values to words according to their importance in the data set.

The preprocessed data set consisted of around 415,294 records. We used the TF-IDF vectorizer to extract 5,000 features from the textual data. Lastly, the data set was split into training and testing data, with 80% allocated for training and 20% allocated for testing and evaluation.

4 DATA PREPROCESSING STEPS

4.1 UPLOADING DATASET INTO GOOGLE COLAB:

```
from google.colab import files

uploaded = files.upload()
```

```
Choose Files sentiment_analysis.csv
sentiment_analysis.csv(text/csv) - 51584070 bytes, last modified: 5/31/2026 - 100% done
Saving sentiment_analysis.csv to sentiment_analysis.csv
```

4.2 LOADING DATASET:

```
import pandas as pd

df = pd.read_csv("sentiment_analysis.csv")
```

4.3 DATA NORMALIZATION:

4.3.1 Checking dataset head:

```
df.head()
```

	text	source	label
0	i just feel really helpless and heavy hearted	emotions_processed.csv	4
1	i have enjoyed being able to slouch about rela...	emotions_processed.csv	0
2	i gave up my internship with the dmrg and am f...	emotions_processed.csv	4
3	i do not know i feel so lost	emotions_processed.csv	0
4	i am a kindergarten teacher and i am thoroughl...	emotions_processed.csv	4

4.3.2 Checking dataset column:

```
print(df.columns)
```

```
Index(['text', 'source', 'label'], dtype='object')
```

4.3.3 Checking dataset shape:

```
print(df.shape)
```

```
(416809, 3)
```

4.3.4 Checking Missing Values:

```
print(df.isnull().sum())
```

```
text      0
source    0
label     0
dtype: int64
```

4.3.5 Removing Missing values if any:

```
df = df.dropna()

print(df.shape)
```

```
(416809, 3)
```

4.3.6 Removing Duplicates:

```
print("Before:", df.shape)

df = df.drop_duplicates()

print("After:", df.shape)
```

```
Before: (416809, 3)
After: (415294, 3)
```

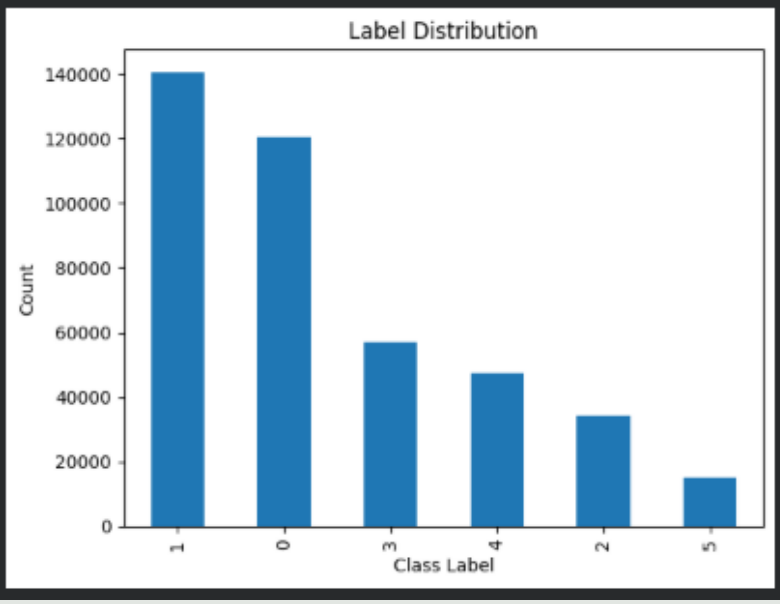
4.3.7 Checking label distribution:

```
print(df['label'].value_counts())
```

```
label
1    140543
0    120707
3     57103
4     47562
2     34448
5     14931
Name: count, dtype: int64
```

4.3.8 Visualizing Label distribution:

```
import matplotlib.pyplot as plt

df['label'].value_counts().plot(kind='bar')

plt.title("Label Distribution")
plt.xlabel("Class Label")
plt.ylabel("Count")

plt.show()
```



4.3.9 Preparing features & targets:

```
X = df['text']

y = df['label']
```

4.3.10 TD-IDF feature extraction:

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(
    max_features=5000,
    stop_words='english'
)

X = tfidf.fit_transform(X)

print(X.shape)
```

```
(415294, 5000)
```

4.3.11 Train/test split:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print(X_train.shape)
print(X_test.shape)
```

```
(332235, 5000)
(83059, 5000)
```

5 MACHINE LEARNING MODELS USED

5.1 MODEL TRAINING:

5.1.1 Logistic Regression:

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

lr = LogisticRegression(max_iter=1000)

lr.fit(X_train, y_train)

pred_lr = lr.predict(X_test)

acc_lr = accuracy_score(y_test, pred_lr)

print("Logistic Regression Accuracy:", acc_lr)
```

```
Logistic Regression Accuracy: 0.8960618355626723
```

5.1.2 Naïve Bayes

```
from sklearn.naive_bayes import MultinomialNB

nb = MultinomialNB()

nb.fit(X_train, y_train)

pred_nb = nb.predict(X_test)

acc_nb = accuracy_score(y_test, pred_nb)

print("Naive Bayes Accuracy:", acc_nb)
```

```
Naive Bayes Accuracy: 0.8510456422543011
```

5.1.3 Decision Tree

```
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(
    max_depth=20,
    random_state=42
)

dt.fit(X_train, y_train)

pred_dt = dt.predict(X_test)

acc_dt = accuracy_score(y_test, pred_dt)

print("Decision Tree Accuracy:", acc_dt)
```

```
Decision Tree Accuracy: 0.4016301665081448
```

5.1.4 Support vector Machine

```
from sklearn.svm import LinearSVC

svm = LinearSVC()

svm.fit(X_train, y_train)

pred_svm = svm.predict(X_test)

acc_svm = accuracy_score(y_test, pred_svm)

print("SVM Accuracy:", acc_svm)
```

```
SVM Accuracy: 0.8949060306529094
```

6 MODEL ACCURACY AND EVALUATION

6.1 MODEL EVALUATION

```python
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

models = {
    "Logistic Regression": pred_lr,
    "Naive Bayes": pred_nb,
    "SVM": pred_svm,
    "Decision Tree": pred_dt
}

for name, pred in models.items():

    print(f"\n{name}")

    print("Accuracy :", accuracy_score(y_test, pred))

    print("Precision:",
          precision_score(y_test, pred, average='weighted'))

    print("Recall   :",
          recall_score(y_test, pred, average='weighted'))

    print("F1-Score :",
          f1_score(y_test, pred, average='weighted'))
```

```
Logistic Regression
Accuracy : 0.8960618355626723
Precision: 0.8952679146701499
Recall   : 0.8960618355626723
F1-Score : 0.8954941516428233

Naive Bayes
Accuracy : 0.8510456422543011
Precision: 0.862939626710179
Recall   : 0.8510456422543011
F1-Score : 0.8412806674302352

SVM
Accuracy : 0.8949060306529094
Precision: 0.8945043462964117
Recall   : 0.8949060306529094
F1-Score : 0.8946509388196501

Decision Tree
Accuracy : 0.4016301665081448
Precision: 0.5332446729526599
Recall   : 0.4016301665081448
F1-Score : 0.2788726650636339
```

6.1.1 Evaluation Results

	Model	Accuracy	Precision	Recall	F1-Score
0	Logistic Regression	0.896062	0.895268	0.896062	0.895494
1	Naive Bayes	0.851046	0.862940	0.851046	0.841281
2	SVM	0.894906	0.894504	0.894906	0.894651
3	Decision Tree	0.401630	0.533245	0.401630	0.278873

Next steps: Generate code with evaluation | New interactive sheet

From the above results, it can be observed that Logistic Regression outperformed all others with high accuracy, followed by SVM. Naive Bayes gave reasonable output results, while Decision Tree performed worst among others.

The high accuracy of Logistic Regression and SVM demonstrates that these algorithms are well-suited for text classification tasks involving mental health conversations and sentiment analysis.

6.2 Model Accuracy

Accuracy was used to determine the performance of all machine learning models. Training data consisted of 80% of the entire dataset and testing data was 20%.

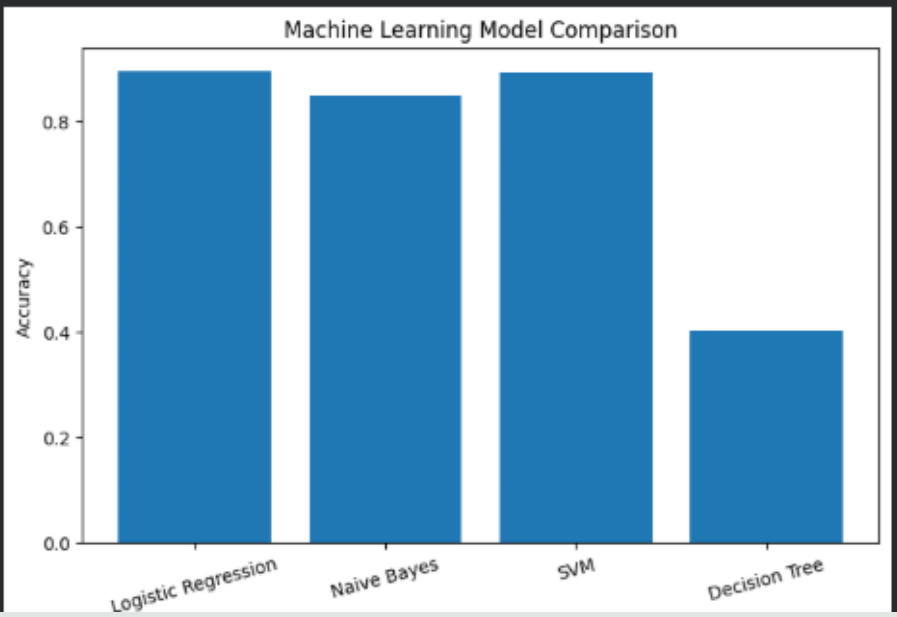
```
import pandas as pd

results = pd.DataFrame({
    'Model': [
        'Logistic Regression',
        'Naive Bayes',
        'SVM',
        'Decision Tree'
    ],
    'Accuracy': [
        acc_lr,
        acc_nb,
        acc_svm,
        acc_dt
    ]
})

print(results)
```

```
                Model  Accuracy
0  Logistic Regression  0.896062
1          Naive Bayes  0.851046
2                  SVM  0.894906
3        Decision Tree  0.401630
```

7 COMPARISON OF MODELS

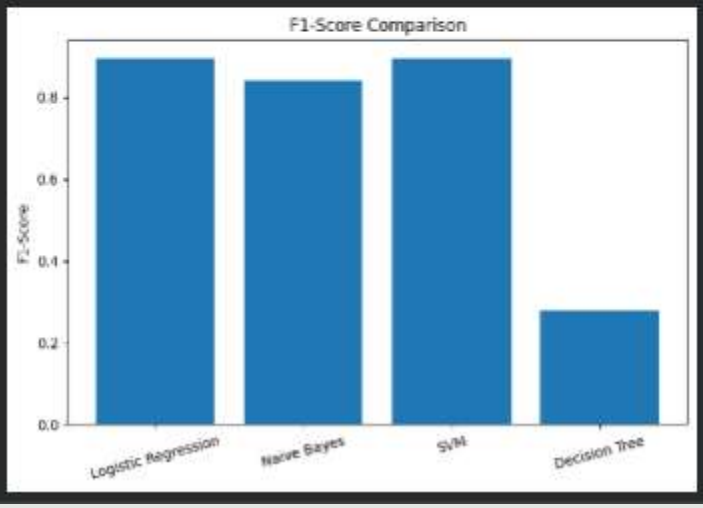
7.1 GRAPH 1: ACCURACY COMPARISON

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))
plt.bar(results['Model'], results['Accuracy'])

plt.title('Machine Learning Model Comparison')
plt.xlabel('Models')
plt.ylabel('Accuracy')

plt.xticks(rotation=15)

plt.show()
```



7.2 GRAPH 2: F1-SCORE COMPARISON:

```
plt.figure(figsize=(8,5))

plt.bar(evaluation['Model'],
        evaluation['F1-Score'])

plt.title("F1-Score Comparison")

plt.ylabel("F1-Score")

plt.xticks(rotation=15)

plt.show()
```



8 BEST MODEL SELECTION

```
best_model = evaluation.loc[
    evaluation['Accuracy'].idxmax()
]

print("Best Model Selected:")
print(best_model)
```

```
Best Model Selected:
Model        Logistic Regression
Accuracy                0.896062
Precision               0.895268
Recall                  0.896062
F1-Score                0.895494
Name: 0, dtype: object
```

Among all the trained models, the best result was given by Logistic Regression with 89.61% accuracy.

Rationale behind choosing Logistic Regression:

- Highest accuracy score.
- Fast training time.
- Effective performance on large text datasets.

Therefore, Logistic Regression was selected as the best-performing model for the Mental Health Chatbot project.

9 EXAMPLE

```
while True:
    text = input("You: ")

    if text.lower() == "exit":
        break

    text_vector = tfidf.transform([text])

    prediction = lr.predict(text_vector)

    print("Predicted Mental Health Category:", prediction[0])
```

```
You: i am sad
Predicted Mental Health Category: 0
You: i am dying
Predicted Mental Health Category: 0
You: i am happy
Predicted Mental Health Category: 1
You:
```

In dataset, **category = label/class** assigned to a mental health statement.

10 CONCLUSION

The current study has been successful in applying machine learning algorithms for classifying the mental health-related text conversations. Four machine learning models were trained and tested after pre-processing the data as well as transforming the text data into numbers using the TF-IDF algorithm.

From the results of experiments conducted, it is evident that logistic regression provided the highest accuracy rate of 89.61% while support vector machine provided accuracy of 89.46%. The current system clearly indicates the applicability of machine learning algorithms in mental health apps and conversational AI.

Future improvements may include deep learning, transformer architectures, as well as live implementation of the chatbot.

11 REFERENCES

- Kaggle – Mental Health Conversational AI Training Dataset
- Scikit-Learn Documentation
- Pandas Documentation
- NumPy Documentation

- Matplotlib Documentation
- Python Official Documentation